Process context frame

Relevant files

resource-allocation-process.txt

Chat notes

Resource Allocation

Resource Allocation
Owner: Kofi
Last Updated: Aug 23, 2026
Scope: The Resource Allocation flowchart covers two connected algorithms: greedy assignment of pending requests to compatible available resources, followed by budget-constrained request selection using dynamic programming. It begins with request/resource lists and ends with assignments plus the maximum achievable value and chosen requests.

Diagram key
- ▭ Process
- ◆ Decision

Sub-process A — Greedy Assignment
Start: list of pending requests and list of available resources
Sort pending requests by urgency, highest first
Set current request = first request in the sorted list
Are there more requests to process?
No
Return all assignments
Yes
Determine compatible resource types for the request category
Scan resources in order for the first available compatible resource
Was a compatible available resource found?
Yes
Mark resource unavailable; record assignment
No
Record the request as unassigned
Move to next request
Note: greedy is fast but not always optimal — see documented counterexample in report.
Proceed to budget-constrained selection
Sub-process B — Budget-Constrained Selection (Dynamic Programming)
Start: list of requests each with a cost and a value, and a fixed budget
Create a 2D table for requests and budget amounts, initialized to 0
Set row i = 1
Is row i less than or equal to the number of requests?
No
Reconstruct chosen requests by walking backward through the table
Return the maximum total value achievable, and the list of chosen requests
Yes
Set column b = 0
Is column b less than or equal to the budget?
Yes
Does request i's cost fit within budget b?
No
Set table[i][b] = table[i-1][b]
Yes
Set table[i][b] to the larger of skip or include value
Increment column b by 1
No
Increment row i by 1

Request Assignment and Budget-Constrained Selection

Diagram information

This diagram presents a two-stage process. Sub-process A greedily assigns pending requests to compatible available resources in urgency order, recording unassigned requests when no resource is available. The resulting assignments then proceed to Sub-process B, which uses dynamic programming to select requests that maximize total value within a fixed budget and returns the optimal selection.

Request Assignment and Budget-Constrained Selection SOP

1. Start with the list of pending requests and available resources.
2. Sort pending requests by urgency, with the highest urgency first.
3. Set the current request to the first request in the sorted list.
4. Determine the resource types compatible with the current request category.
5. Scan resources in order for the first available compatible resource.
6. Determine whether a compatible available resource was found.
    6.1. If a compatible resource is found, mark it as unavailable and record the assignment.
    6.2. If no compatible resource is found, record the request as unassigned.
7. Move to the next request.
8. Determine whether more requests remain to be processed.
    8.1. If more requests remain, repeat the compatibility and resource-scanning steps.
    8.2. If no requests remain, return all assignments.
9. Note that the greedy assignment method is fast but may not produce an optimal result; consult the documented counterexample in the report.
10. Proceed to budget-constrained selection.
11. Start with requests containing a cost and value, together with a fixed budget.
12. Create a two-dimensional table for requests and budget amounts, initialized to zero.
13. Set row i to 1.
14. Determine whether row i is less than or equal to the number of requests.
    14.1. If no, reconstruct the chosen requests by walking backward through the table.
    14.2. Return the maximum achievable total value and the list of chosen requests.
    14.3. If yes, set column b to 0.
15. Determine whether column b is less than or equal to the budget.
    15.1. If no, increment row i by 1 and repeat the row-processing check.
    15.2. If yes, evaluate whether request i's cost fits within budget b.
16. If the request cost fits within budget b, set table[i][b] to the larger of the skip value and include value.
17. If the request cost does not fit within budget b, set table[i][b] equal to table[i-1][b].
18. Increment column b by 1 and continue processing the budget columns.
19. After all rows and budget columns are processed, reconstruct the selected requests from the completed table.
20. Return the maximum total value achievable and the corresponding list of selected requests.

Generated by ***@Kofi*** on August 23, 2026